HIGH PERFORMANCE DATA PIPELINE

$0.00 LOCAL OPEX

Movielens 25M+ Rating Data Relational Inner Join Pipeline

Low-latency relational join processing across **25M+** ratings and genome data using R Tidyverse on constrained local hardware.

DATASET SCALE

25,000,000+ Rows

HARDWARE SPEC

Core i3 @1.2GHz

OPEX COST

$0.00 Cloud OPEX

SYSTEM ARCHITECTURE

System Environment & Hardware Specs

Operating under severe local resource constraints with zero cloud dependencies.

CPU STICKER

Core i3 @1.2GHz

Single-threaded bound. Requires explicitly vectorized R sweeps.

RAM BADGE

4.0 GB RAM Ceiling

Strict RAM ceiling with zero room for intermediate data copies.

~2.0 GB OS Buffer

Win 10 OS overhead leaves an active 1.8 GB memory buffer.

No Cloud Support

Runs entirely local with $0.00 cloud OPEX expenditure.

Constraint Strategy: Stream parsing + early garbage collection

✓ 100% Local Execution

EXECUTION PIPELINE

System Architecture – Data Pipeline Flow

Four modular execution stages optimized for low memory footprint.

Ingestion

Streams **25M+** ratings & genome data via readr chunked parsing.

Cleaning

Deduplicates upfront, casts IDs to 32-bit integers, and prunes anomalies.

Join

Aggregates genome tags & executes C++ inner join on movieId.

Export

Exports cleaned CSV dataset to disk disabling row indices.

Pipeline Architecture engineered for zero-crash operations under **~1.8 GB** active RAM ceiling.

STAGE 1 INGESTION

Stage 1: R Data Ingestion (25M+ Rows)

Stage1_Ingestion.R

Stage1_Ingestion.R — Fast Stream Parsing

```r
library(tidyverse); library(readr)
# Ingest Ratings & Genome Raw Streams
rating_data ← read_csv("ratings.csv")
glimpse(rating_data)
genome_data ← read_csv("genome-scores.csv")
glimpse(genome_data)
```

Fast Stream Reading

Employs chunk-based memory parsing for rapid ingestion of large text files.

Structure Inspection

Invokes glimpse() to audit raw column types safely prior to joining.

Zero Memory Bloat

Optimized buffer keeps allocation under the strict 1.8GB active limit.

STAGE 2 CLEANING

Stage 2: Data Cleaning & Sanitization

Stage2_Cleaning.R

Stage2_Cleaning.R — Upfront Sanitization

```r
cleaned_rating_data ← rating_data %>%
  distinct() %>%
  mutate(
    userId = as.integer(userId),
    movieId = as.integer(movieId),
    rating = as.numeric(rating)
  ) %>%
  filter(userId > 0 & movieId > 0 & rating > 0.00)
```

Upfront Deduplication

Applies distinct() early to drop duplicate records before processing.

Strict Type Casting

Casts IDs via as.integer() reducing memory footprint significantly.

Noise Pruning

Filters non-positive ratings to purge corrupt entries upfront.

STAGE 3 JOINING

Stage 3: Genome Compression & Relational Join

Stage3_Joining.R

Stage3_Joining.R — C++ Hash Inner Join

```
cleaned_genome_data ← genome_data %>%
  distinct() %>%
  mutate(movieId = as.integer(movieId), tagId = as.integer(tagId)) %>%
  group_by(movieId) %>%
  summarise(Total_Tags = n(), .groups = 'drop')

join_report ← cleaned_rating_data %>%
  inner_join(cleaned_genome_data, by = 'movieId')
```

Genome Compression

Groups million-row tags by movieId into concise summary counts.

C++ Hash Inner Join

Executes inner_join() using optimized C++ hash routines for speed.

RAM Thrashing Control

Prevents disk swapping under the strict 1.8 GB active memory limit.

STAGE 4 PERSISTENCE

Stage 4: Persistence & Disk Export

</> Stage4_Persistence.R

Stage4_Persistence.R — BI Export

```
# Export cleaned joining pipeline dataset
final_report ← join_report %>%
  distinct() %>%
  select(userId, movieId, rating, Total_Tags)

write.csv(
  final_report,
  "New_dataset_movielens_25M_pipeline.csv",
  row.names = FALSE
)
```

Row Index Removal

Disables redundant row index numbers via row.names = FALSE.

BI-Ready CSV Export

Outputs structured CSV file ready for instant Tableau or Power BI loading.

Validated Persistence

Full 25M+ dataset successfully joined and saved to disk.

PIPELINE ARCHITECT & DEVELOPER

Connect & Code Repository

Thank you for reviewing the MovieLens 25M R Relational Pipeline Architecture. Access full source code & profiles below!



LinkedIn Profile

www.linkedin.com/in/niaz-shoyab-495061b8



GitHub Repository

www.github.com/talkmridul-bit

MovieLens 25M Relational ETL Pipeline • High Performance R Engineering